Entrega Escalável de Vídeo Volumétrico Baseada em MDC para Conferências WebRTC Um-para-Muitos em Tempo Real

Matthias De Fré · Jeroen van der Hooft · Tim Wauters · Filip De Turck
IDLab, Ghent University – imec, Gante, Bélgica

MMSys '24 (ACM Multimedia Systems Conference), 15–18 de abril de 2024, Bari, Itália. DOI: 10.1145/3625468.3647617

Tradução não oficial para o português (pt-BR) de *Scalable MDC-Based Volumetric Video Delivery for Real-Time One-to-Many WebRTC Conferencing*. Termos técnicos consagrados (como *bitrate*, *throughput*, *jitter*) foram mantidos em inglês quando soam mais naturais assim. Figuras e imagens foram omitidas (permanecem apenas as legendas traduzidas); tabelas e equações foram reproduzidas. A lista de referências é mantida no idioma original. Em caso de dúvida, vale o texto original.

RESUMO

A produção e o consumo de conteúdo em vídeo tornaram-se um item básico nos dias de hoje. Com a ascensão da realidade virtual (VR), os usuários buscam experiências imersivas e interativas que combinem aplicações clássicas de vídeo, como conferências ou shows digitais, com tecnologias mais novas. Ir além do vídeo 2D rumo a uma experiência de 360 graus foi o primeiro passo. No entanto, um vídeo de 360 graus oferece apenas movimento rotacional, o que dificulta a interação com o ambiente. Formatos de conteúdo 3D totalmente imersivos, como campos de luz (*light fields*) e vídeo volumétrico, aspiram a ir além, permitindo seis graus de liberdade (6DoF), isto é, liberdade tanto rotacional quanto posicional. Mesmo assim, a adoção de métodos de captura e renderização de vídeo imersivo tem sido prejudicada por seus substanciais requisitos de largura de banda e de computação, o que os torna, na maioria dos casos, impraticáveis para aplicações de baixa latência. Diversos esforços foram feitos para atenuar esses problemas, com a introdução de algoritmos de compressão especializados e o uso de métodos de adaptação 2D existentes para adaptar a qualidade com base na largura de banda disponível ao usuário. Entretanto, embora esses métodos melhorem a qualidade de experiência (QoE) e as limitações de largura de banda, eles ainda sofrem de alta latência, o que inviabiliza a interação em tempo real. Para tratar essa questão, apresentamos uma nova arquitetura de streaming um-para-muitos, open source [32], que usa vídeo volumétrico baseado em nuvens de pontos. Para reduzir os requisitos de largura de banda, utilizamos o codec Draco para comprimir as nuvens de pontos antes de transmiti-las por WebRTC, o que garante baixa latência e viabiliza o streaming de vídeo volumétrico interativo 6DoF em tempo real. O conteúdo é adaptado por meio de uma estratégia de codificação por múltiplas descrições (MDC), que combina descrições amostradas da nuvem de pontos com base na largura de banda estimada retornada pelo algoritmo Google Congestion Control (GCC). A codificação MDC escala mais facilmente para um número maior de usuários do que a codificação individual. A solução proposta alcança latência em tempo real semelhante para três e nove clientes (163 ms e 166 ms), o que é 9% e 19% menor que na codificação individual. A abordagem baseada em MDC, usando três workers, alcança qualidade visual semelhante à de uma solução com codificação por cliente usando cinco threads de trabalho, e qualidade superior quando o número de clientes é maior que 20.

Palavras-chave: Vídeo volumétrico, streaming adaptativo, WebRTC, conferência virtual, codificação por múltiplas descrições, realidade virtual.

Conceitos CCS: Sistemas de informação → Streaming multimídia; Computação centrada no ser humano → Realidade virtual

1 INTRODUÇÃO

A crescente demanda por ferramentas de comunicação remota, como Microsoft Teams e Zoom, vem recebendo maior atenção nos últimos tempos [20]. Ainda assim, essas plataformas oferecem uma experiência interativa restrita, pois permitem ver os demais participantes apenas por meio de imagens bidimensionais. Foram feitas propostas de implementações de vídeo imersivo com seis graus de liberdade (6DoF) que permitem aos usuários interagir com liberdade rotacional e posicional [1]. No contexto dessas aplicações, deve-se fazer uma distinção com base nos requisitos de interação dos usuários. Em interações um-a-um, como chamadas de vídeo, um requisito importante é ter a menor latência possível para permitir interação em tempo real e evitar desconforto ao conversar com o outro usuário [38]. Em conferências virtuais, com muitos remetentes e receptores, surge a demanda adicional de conseguir adaptar o conteúdo de vídeo para um grande número de usuários mantendo baixa latência [25]. Aplicações um-para-muitos, como uma sala de aula virtual no contexto da educação ou um show virtual no contexto do entretenimento, representam uma categoria híbrida em que, em vez de vários participantes gerarem conteúdo, um único par atua como fonte do conteúdo, simplificando o projeto do sistema necessário para garantir baixa latência. Escolher a forma correta de comunicação é importante para garantir que os requisitos de interatividade sejam atendidos, além de ter uma taxa de bits e uma latência aceitáveis.

Existem vários formatos para viabilizar o uso de conteúdo 6DoF em aplicações imersivas, comumente renderizados e exibidos em um headset (HMD, *head-mounted display*). Esses formatos podem ser categorizados em duas classes de vídeo imersivo 6DoF: baseado em imagens ou volumétrico [46]. As técnicas de vídeo baseadas em imagens, como os campos de luz, aproveitam uma grande quantidade de imagens para gerar uma nova imagem renderizada com base no ângulo de visão do usuário. A captura é feita ou por uma única câmera com uma multiplicidade de lentes, conhecida como câmera *lenslet*, ou por uma configuração avançada de câmeras com dezenas ou centenas delas [7]. A escolha da configuração de câmeras depende dos requisitos posicionais da aplicação. A necessidade de muitas câmeras decorre da exigência de gerar qualquer combinação possível de posição e inclinação ao assistir ao vídeo [39]. Em contraste, o vídeo volumétrico se baseia no uso de representações tridimensionais, como malhas (*meshes*) e nuvens de pontos, para retratar objetos em um ambiente 3D. As malhas são carac-

terizadas por vértices interligados por arestas que formam os polígonos usados para renderizar o objeto no ambiente. Já as nuvens de pontos consistem exclusivamente de vértices. Além disso, em vez de usar mapeamento de textura, uma nuvem de pontos usa um atributo de cor para cada ponto [5]. Essas representações são então renderizadas em uma cena virtual, o que permite visualizá-las de diferentes ângulos e posições. Assim como no vídeo baseado em imagens, é necessária uma configuração especializada de câmeras para capturar a representação 3D do objeto. No caso das nuvens de pontos, várias câmeras são posicionadas com precisão ao redor do objeto para capturá-lo de diferentes ângulos. Essa configuração produz várias nuvens de pontos distintas, que são então combinadas (*stitching*) para remover os pontos duplicados e manter um único objeto como resultado final [2]. Em comparação com o vídeo de campo de luz, essa abordagem reduz substancialmente a largura de banda necessária para transmissão, pois não é mais preciso manter uma grande coleção de imagens para cada quadro; basta transferir um único objeto 3D. Além disso, os requisitos computacionais associados à renderização de vídeo volumétrico são significativamente menores que os das soluções baseadas em imagens, o que é uma necessidade para viabilizar renderização e interação em tempo real no lado do cliente.

Apesar da redução substancial da largura de banda necessária, as taxas de bits resultantes ainda são, muitas vezes, altas demais para as infraestruturas de rede atuais. O dataset de nuvens de pontos 8i [12] consiste em vários vídeos de nuvens de pontos em alta resolução, cada um exigindo uma taxa de dados considerável, de até 5,2 Gb/s, que só pode ser usada em ambientes muito especializados, com conexões dedicadas de largura de banda ultra-alta [46]. Mecanismos adicionais, como compressão ou adaptação, são imprescindíveis para permitir o uso desse conteúdo imersivo por uma variedade de usuários com diferentes condições de rede e restrições de largura de banda. Os HMDs são capazes de rastrear tanto a posição quanto o campo de visão (FoV, *field of view*), seja por meio dos sensores internos do headset, o que se chama de rastreamento *inside-out*, seja por dispositivos externos posicionados no ambiente do usuário, o que se chama de rastreamento *outside-in* [18]. Aproveitando o FoV e os dados de posição do usuário, torna-se possível adaptar o conteúdo de vídeo dinamicamente, transmitindo e renderizando apenas os detalhes importantes do vídeo, o que resulta em uso de largura de banda significativamente menor.

Um componente importante para garantir a comunicação em tempo real é a escolha do protocolo de transporte. Soluções baseadas em TCP, como o HLS de baixa latência (LL-HLS) e o DASH de baixa latência (LL-DASH), não oferecem a latência de tempo real exigida para conferências virtuais um-para-muitos e muitos-para-muitos, por causa da sobrecarga introduzida pelos mecanismos forçados de entrega confiável, como confirmações (*acknowledgments*), controle de fluxo, retransmissões e reordenação de pacotes. Soluções baseadas em UDP, como o WebRTC, buscam desempenho em tempo real trocando recursos do TCP, como a entrega garantida, por uma latência significativamente menor [3].

Este artigo propõe um novo pipeline de ponta a ponta projetado para aplicações um-para-muitos, com as seguintes contribuições:

Figura 1: Uma arquitetura de streaming de vídeo volumétrico. *(imagem omitida nesta tradução; consulte o artigo original)*

- Uma solução escalável de adaptação de qualidade por codificação por múltiplas descrições (MDC), que permite streaming de maior qualidade para um grande número de clientes ($n > 20$) em comparação com uma solução de codificação por cliente;
- Um pipeline da captura à renderização que emprega WebRTC, adaptado para dar suporte a streaming de nuvens de pontos, com comunicação em tempo real e latência média de captura à exibição de 163 ms;
- Um algoritmo adaptativo de alocação de taxa de bits, que garante que cada cliente receba uma representação de qualidade adequada às suas condições de rede e à sua posição no ambiente virtual;
- Uma análise comparativa entre a abordagem baseada em MDC proposta e uma solução de codificação otimizada por cliente, avaliando seu desempenho em termos de qualidade e escalabilidade.

O restante do artigo está organizado da seguinte forma. A Seção 2 apresenta uma visão geral do estado da arte em streaming de vídeo volumétrico. A Seção 3 introduz a abordagem proposta para viabilizar o streaming adaptativo de vídeo volumétrico. A Seção 4 apresenta nossa metodologia de avaliação, seguida dos resultados e da discussão de nossos experimentos. Por fim, a Seção 5 conclui o artigo com uma breve visão geral dos resultados mais importantes e de possíveis trabalhos futuros.

2 TRABALHOS RELACIONADOS

Esta seção apresenta uma visão geral da literatura relacionada aos componentes necessários para estabelecer uma arquitetura de streaming de vídeo volumétrico um-para-muitos em tempo real, conforme ilustrado na Figura 1. Primeiro, descrevemos as possíveis representações de vídeo volumétrico e os métodos de captura, seguidos de uma visão geral dos codecs de nuvem de pontos do estado da arte. Para cada um desses codecs, discutimos as vantagens e desvantagens quanto à sua aplicação em um pipeline de ponta a ponta. Em seguida, explicamos os mecanismos de entrega de baixa latência, a adaptação de qualidade e as métricas de qualidade, tanto no contexto do vídeo tradicional quanto do vídeo volumétrico.

2.1 Vídeo volumétrico e métodos de captura

Tanto as malhas quanto as nuvens de pontos são usadas como fonte de conteúdo de vídeo imersivo, cada uma com suas próprias vantagens e desvantagens. As malhas aproveitam certos recursos e otimizações do pipeline gráfico, como a filtragem anisotrópica, para aumentar sua qualidade visual e desempenho [51]. No entanto, as nuvens de pontos empregam algoritmos de *culling* e *tiling* melhores e otimizados, por dependerem puramente de vértices [8]. Além disso, o processo de captura é mais simples com nuvens de pontos, já que os pontos capturados e suas cores são usados diretamente. Em contraste, as malhas exigem que os pontos capturados sejam convertidos em uma topologia geométrica antes da renderização; esse processo consome tempo e afeta negativamente a latência do sistema [11]. No restante deste artigo, o foco será o uso de nuvens de pontos como fonte de vídeo volumétrico.

As câmeras LiDAR usam lasers refletidos para obter uma representação 3D altamente precisa de uma área grande, ao custo de maior consumo de energia [15]. Em contraste, os sistemas baseados em profundidade empregam um sensor estereoscópico para estimar a profundidade de cada pixel de uma imagem de profundidade [28]. Para vídeo volumétrico, é comum usar câmeras de profundidade devido ao seu menor custo e consumo de energia.

2.2 Compressão de nuvens de pontos e codecs

A compressão de nuvens de pontos reduz significativamente a largura de banda necessária; podem-se distinguir duas categorias de codecs de compressão: baseados em projeção e baseados em geometria. Os codecs baseados em projeção, como o V-PCC, projetam a nuvem de pontos em um plano 2D, permitindo o uso de codecs de vídeo 2D existentes no plano de projeção. Já os codecs geométricos, como o Draco ou o G-PCC, são projetados para codificar a nuvem de pontos como uma estrutura 3D [19].

O V-PCC emprega projeções 2D da nuvem de pontos para criar uma imagem contendo as diferentes vistas projetadas. Em seguida, essa imagem é codificada usando codecs de vídeo existentes [19]. Esse método alcança uma alta taxa de compressão ao custo de velocidades de codificação fora do tempo real [50]. Além disso, o V-PCC suporta predição entre quadros (*inter-frame*), que agrupa quadros consecutivos em um segmento. No entanto, a implementação utilizada tem pouco ou nenhum impacto nas taxas de bits resultantes e, em vez disso, aumenta consideravelmente a latência [14].

Em contraste com o V-PCC, os codificadores geométricos utilizam estruturas de dados 3D, como octrees e kd-trees, para codificar a nuvem de pontos. Entre os codificadores geométricos, apenas o Draco alcança codificação em tempo real ao usar nuvens de pontos grandes, de mais de 750.000 pontos [14].

2.3 Entrega de baixa latência

Dado que o vídeo volumétrico é fundamentalmente uma extensão do vídeo convencional, é viável adaptar soluções existentes de streaming de vídeo 2D para incorporar os recursos necessários para suportar vídeo volumétrico. Tipicamente, implementações baseadas em HTTP são consideradas por suas transmissões confiáveis, que são um pré-requisito para a maioria das aplicações [13]. O streaming adaptativo HTTP (HAS, *HTTP adaptive streaming*) representa uma abordagem amplamente adotada para obter reprodução de vídeo 2D adaptativa. Formatos de streaming baseados em HAS, como DASH e HLS, dividem o conteúdo de vídeo em segmentos codificados em múltiplas representações de qualidade com taxas de bits-alvo variadas [41]. Esse processo introduz uma latência entre 5 e 18 segundos [49], o que o torna inadequado para streaming em tempo real. No entanto, mesmo as variantes de baixa latência, como o LL-DASH e o LL-HLS, só conseguem atingir uma latência entre 1 e 5 segundos [49], o que ainda é inadequado [6]. Os formatos de streaming mencionados utilizam TCP, o que aumenta ainda mais a latência devido à sobrecarga criada pelos mecanismos do TCP, como o handshake ou a confiabilidade forçada [9]. A comunicação web em tempo real (WebRTC) é uma solução baseada em UDP, projetada especificamente para viabilizar comunicação interativa em tempo real [27]. Em comparação com outros formatos de streaming, o WebRTC alcança atraso inferior a um segundo [49]. Embora o WebRTC tenha sido projetado como uma solução ponto a ponto (*peer-based*), o que apresenta desafios de escalabilidade, foram projetadas várias arquiteturas cliente-servidor para permitir transmissões de baixa latência em larga escala. Arquiteturas como a unidade de controle multiponto (MCU) e a unidade de encaminhamento seletivo (SFU) empregam um servidor central responsável por adaptar a qualidade para cada usuário com base em sua largura de banda. Por essas razões, usaremos o WebRTC em uma arquitetura cliente-servidor para viabilizar streaming adaptativo de baixa latência em um ambiente com múltiplos clientes.

2.4 Adaptação de qualidade

O conceito de adaptação de qualidade é uma prática consolidada no streaming convencional para garantir reprodução de vídeo fluida sem perda de pacotes [37]. A maioria dos codificadores gera várias representações de qualidade a partir de taxas de bits-alvo variadas [4]. No contexto de um cenário um-a-um, a taxa de bits fornecida pode ser uma estimativa da largura de banda disponível ao usuário. No entanto, para um número maior de usuários, torna-se inviável ter codificação individual; em vez disso, várias taxas de bits padrão são cuidadosamente escolhidas para garantir uma baixa latência e um fluxo de vídeo de qualidade aceitável para cada usuário. Alternativamente, em vez de empregar níveis de qualidade separados, pode-se adotar uma abordagem MDC. Na adaptação baseada em MDC, o conteúdo de vídeo é codificado em descrições individuais que são combinadas para alcançar níveis variados de qualidade. Embora esses métodos de adaptação tenham sido projetados para vídeo 2D, certos aspectos podem ser adaptados e integrados a cenários de vídeo imersivo.

No vídeo volumétrico, o usuário pode alterar seu ponto de vista, fazendo objetos saírem do ponto de vista atual e tornando ineficiente continuar transmitindo esses objetos. van der Hooft et al. [48] codificam cada nuvem de pontos da cena em várias representações de qualidade. É proposto um algoritmo de adaptação de qualidade que usa o FoV e a posição do usuário para determinar qual representação de cada objeto deve ser transmitida para alcançar a qualidade ótima para a largura de banda disponível. No entanto, propõe-se uma abordagem baseada em DASH, em que os quadros são concatenados em segmentos, o que a torna inadequada para comunicação em tempo real. Uma abordagem baseada em MDC atenua esse problema, pois permite a criação de mais níveis de qualidade, com menos tempo de processamento. Contudo, atualmente não existe uma solução de streaming de vídeo volumétrico baseada em MDC que utilize WebRTC.

Para adaptar a qualidade, é necessária uma estimativa precisa da largura de banda. Em comparação com o TCP, o UDP carece inerentemente de controle de fluxo e de um mecanismo de estimativa de largura de banda. Comumente, a estimativa de largura de banda é implementada na camada de aplicação. Controladores de congestionamento, como o Google Congestion Control (GCC) [10] e o Self-Clocked Rate Adaptation for Multimedia (SCReAM) [26], são implementados na aplicação multimídia e exigem a troca de mensagens de controle entre remetente e receptor para estimar a largura de banda disponível. Por essa razão, utilizaremos o GCC para implementar a estimativa de largura de banda no WebRTC.

2.5 Renderização de vídeo volumétrico

A maioria das soluções de renderização é otimizada para renderizar malhas com eficiência. As malhas usam as primitivas de

triângulo das interfaces de programação de aplicações (APIs) gráficas para renderizar seus objetos de forma eficiente. Schütz et al. [40] usam primitivas de ponto para renderizar a nuvem de pontos. Além disso, o tamanho dos pontos é alterado para reduzir o número de lacunas, aumentando a qualidade percebida pelo usuário.

Figura 2: A arquitetura do sistema para a solução de streaming de vídeo volumétrico um-para-muitos com codificação individual.
(imagem omitida nesta tradução; consulte o artigo original)

2.6 Métricas de qualidade

A qualidade de um vídeo volumétrico depende de uma multiplicidade de fatores. Em cenários de tempo real, a taxa de quadros pode ser limitada por restrições de largura de banda ou pela latência introduzida pelo codec, o que afeta negativamente a experiência do usuário [53]. A avaliação da qualidade objetiva pode ser feita por meio de métricas de relação sinal-ruído de pico (PSNR), que se dividem em dois tipos. Tian et al. [42] introduzem várias métricas geométricas que avaliam a deformação da malha ou da nuvem de pontos em relação ao objeto original, independentemente de possíveis viewports. Em contraste, Torlig et al. propõem uma abordagem baseada em projeção. Esse método usa múltiplos ângulos para projetar a nuvem de pontos em uma imagem 2D, permitindo o uso de métricas PSNR 2D existentes. Ambos os tipos de PSNR mencionados sofrem da limitação de não conseguir priorizar as seções do objeto que atraem mais atenção do que outras, como as mãos [52], o que dificulta aproximar objetivamente a qualidade percebida pelo usuário.

O Video Multi-Method Assessment Fusion (VMAF) é uma métrica de vídeo que combina as pontuações de várias métricas de avaliação de vídeo para produzir uma única métrica, de 0 a 100, que se correlaciona melhor com a qualidade percebida pelo usuário [36]. Por essa razão, o VMAF será usado neste artigo para avaliar o impacto da abordagem proposta sobre a qualidade visual do vídeo volumétrico.

3 ABORDAGEM PROPOSTA

Esta seção apresenta uma visão geral da arquitetura proposta de streaming de vídeo volumétrico um-para-muitos baseada em MDC. A arquitetura proposta é uma extensão de uma arquitetura simples com codificação individual, da qual um exemplo é mostrado na Figura 2. O objetivo da nossa arquitetura é resolver, em termos de escalabilidade, os problemas que ocorrem ao empregar a arquitetura ilustrada na Figura 2. Nesta seção, descrevemos em detalhe os componentes da arquitetura, suas escolhas de projeto e métodos inovadores.

3.1 Captura com câmera de profundidade

Para a aquisição dos dados da nuvem de pontos, assume-se que uma única fonte de captura registra uma perspectiva frontal de um indivíduo. Utilizando o quadro de profundidade, determinam-se as posições de cada ponto da nuvem. O quadro de cor é usado para mapear um valor RGB aos pontos correspondentes.

3.2 Pré-processamento

Nosso foco principal é o indivíduo, e não o ambiente (estático) ao redor. Assim, será aplicado um filtro de distância rudimentar para descartar os pontos que estão longe demais da câmera. Esse filtro reduz significativamente o número de pontos, permitindo que a largura de banda disponível seja usada com mais eficiência.

Em cenários específicos, uma nuvem de pontos de menor qualidade ou amostrada pode ser suficiente em comparação com a versão em resolução completa. Em particular, isso ocorre ao visualizar objetos distantes, nos quais é mais difícil distinguir os detalhes mais finos [30]. Para simplificar esse processo, empregamos técnicas de amostragem para gerar níveis variados de qualidade da nuvem de pontos.

Para evitar dependências entre quadros, os quadros são codificados exclusivamente por intracodificação (*intra-coding*). Isso garante que cada quadro possa ser decodificado de forma independente, levando a uma qualidade maior quando quadros precisam ser descartados em caso de perda de pacotes.

3.3 Adaptação de qualidade

A etapa inicial do nosso algoritmo de adaptação de qualidade é a criação de diversas representações de qualidade. Para facilitar esse processo, propomos dois métodos distintos capazes de criar essas representações. O objetivo desses métodos é produzir uma representação de qualidade para cada usuário, garantindo ao mesmo tempo que o tempo de pré-processamento permaneça abaixo do tempo entre quadros, para manter o tempo real. Ambos os métodos empregam o princípio da amostragem, que reduz o número de pontos usados no codificador. Propomos usar um amostrador aleatório uniforme, pois ele garante um equilíbrio entre o tempo de amostragem e a qualidade resultante [14].

O primeiro método gera uma representação de qualidade única para cada cliente conectado. Isso é feito calculando primeiro o fator de compressão necessário, igual ao tamanho bruto do quadro dividido pela taxa de bits disponível para aquele quadro. O fator de compressão é então usado junto com o polinômio ajustado da Figura 4 para estimar o fator de amostragem. Esse polinômio foi criado usando um subconjunto aleatório ($n = 600$) da nossa sequência de nuvens de pontos de experimentação, em que cada quadro desse subconjunto foi amostrado repetidamente na faixa de 10% a 100% das nuvens de pontos originais antes da codificação. O polinômio usado depende da resolução das nuvens de pontos codificadas, pois uma nuvem de pontos mais densa afeta as taxas de compressão resultantes. A Figura 2 ilustra a integração desse método em um pipeline de streaming WebRTC. Uma desvantagem substancial desse método é que o número de codificadores necessários é igual ao número de clientes. Se quisermos que cada cliente tenha uma latência de pré-processamento semelhante, precisamos escalar o número de workers, o que é limitado pelos recursos computacionais disponíveis. Além disso, atualizar a arquitetura para dar suporte a um cenário muitos-para-muitos exigiria recursos computacionais adicionais, restringindo ainda mais o número máximo de clientes.

Figura 3: A arquitetura do sistema para a solução proposta de streaming de vídeo volumétrico um-para-muitos baseada em MDC: exige um número fixo de codificadores, de acordo com a quantidade de descrições, e o número de decodificadores locais de cada cliente cresce com o número de descrições recebidas.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 4: O uso de um polinômio ajustado, junto com o fator de compressão necessário, permite determinar a taxa de amostragem necessária (ajuste polinomial de grau 6).
(imagem omitida nesta tradução; consulte o artigo original)

O segundo método, mostrado na Figura 3, segmenta a nuvem de pontos em n descrições, cada uma contendo pontos distintos. A combinação dessas descrições dá acesso a representações de qualidade adicionais. Consequentemente, essa abordagem viabiliza o streaming de $2^n - 1$ combinações possíveis de representações de qualidade. Para gerar as descrições disponíveis, sugerem-se três abordagens:

- **Porcentagem:** Usar n porcentagens fixas para amostrar a nuvem de pontos, criando n descrições que contêm, cada uma, uma fração p_i dos pontos, com i em $1, \ldots, n$, correspondente à respectiva porcentagem;
- **Tamanho fixo:** Limitar o número de pontos a x, sendo x um limite inferior esperado, por meio de amostragem aleatória de x pontos. Em seguida, usar a primeira abordagem para criar n descrições;
- **Taxa de bits fixa:** Aproveitar o polinômio ilustrado na Figura 4 para estimar n porcentagens de amostragem necessárias para criar n descrições com taxas de bits-alvo variadas.

Em comparação com a codificação de vídeo tradicional, que codifica um vídeo em múltiplas representações de qualidade, essas abordagens baseadas em MDC têm melhor desempenho na codificação de nuvens de pontos, devido à ausência de predição entre quadros e à disponibilidade de mais níveis de qualidade com taxas de bits diferentes.

Algoritmo 1 Alocação de taxa de bits usando FoV e posição para calcular as descrições transmitidas.

```
R ← calculateRotationMatrix(yaw, pitch, roll)
viewAngles ← calculateAngles(R)
distance ← getDistanceToUser(userPos, clientPos)
category ← getClientCategory(fov, viewAngles, distance)
descriptions ← getAllowedDescriptions(category)
for d ← descriptions.max() to descriptions.min() do
    if bitrate ≥ getDescriptionSize(d) then
        bitrate ← bitrate − getDescriptionSize(d)
        addDescriptionToFrame(d)
    end if
end for
```

Com o Algoritmo 1, primeiro calculamos a matriz de rotação e os ângulos de visão usando o yaw, o pitch e o roll da câmera. Em seguida, calculamos a distância do usuário ao objeto transmitido. Depois, usamos o ângulo de visão, o FoV e a distância para impor restrições à qualidade máxima alcançável que pode ser transmitida ao cliente. A Figura 5 ilustra o funcionamento desse mecanismo. As representações de alta qualidade só estarão disponíveis para transmissão ao usuário se o objeto estiver próximo e diretamente no FoV do usuário. Além disso, se o objeto estiver fora do FoV ou longe demais do usuário, nenhum quadro será transmitido. Consequentemente, conseguimos calcular as descrições permitidas, das quais um exemplo de implementação é mostrado na Tabela 1. Para a codificação individual, a Tabela 1 mostra a taxa máxima de amostragem permitida e, para as abordagens MDC, indica as combinações permitidas de descrições. Por fim, iteramos sobre as descrições permitidas, começando pela de maior qualidade, e validamos se há largura de banda restante suficiente para adicionar a descrição.

Figura 5: A categoria de qualidade é atribuída com base na distância e no campo de visão (alta qualidade, média qualidade, baixa qualidade e descarte do quadro).
(imagem omitida nesta tradução; consulte o artigo original)

Categoria de qualidade	IDs de descrição usados	% dos pontos na nuvem
Baixa qualidade	1	~15%
Média qualidade	2	~25%
	1, 2	~40%
Alta qualidade	3	~60%
	1, 3	~70%
	2, 3	~85%
	1, 2, 3	~100%

Tabela 1: Exemplo de implementação dos possíveis níveis de qualidade da codificação MDC por meio da combinação de descrições (ID de descrição maior = maior qualidade).

3.4 Entrega baseada em WebRTC

Como buscamos a menor latência possível, utilizamos o WebRTC para garantir comunicação em tempo real entre os componentes de captura e de renderização. Nosso pipeline é voltado a um cenário um-para-muitos, o que resulta em um servidor central no lado da captura que é conectado a cada cliente. Antes do streaming do vídeo, ocorre uma troca do protocolo de descrição de sessão (SDP) entre o servidor e o cliente. Essas mensagens SDP informam à outra aplicação as rotas de rede disponíveis, bem como os codecs suportados. Além disso, como medida para tratar a possível perda de pacotes, os pacotes perdidos são retransmitidos. Isso garante que os quadros permaneçam decodificáveis em caso de perda de pacotes, desde que a perda permaneça dentro de uma certa margem. Atualmente, não há suporte a decodificação e renderização parciais, sendo necessário receber o quadro completo.

Além de transmitir a posição e o FoV, o cliente WebRTC também transmite mensagens de feedback ao servidor, que são então usadas pelo algoritmo de controle de congestionamento do servidor para estimar a largura de banda, permitindo a adaptação de qualidade.

3.5 Interatividade em realidade virtual e renderização

Devido à amostragem que ocorre no algoritmo de adaptação de qualidade, surgirão lacunas nas representações de menor qualidade. Para aumentar a qualidade percebida, aumentamos o tamanho dos pontos com base na razão de amostragem; um exemplo pode ser observado na Figura 6. Para proporcionar uma experiência imersiva, uma solução de renderização simples não é suficiente. Para permitir a interatividade com o ambiente, a nuvem de pontos precisa ser exibida em um HMD. Há dois métodos para isso: renderizar no próprio HMD ou fazer com que um sistema remoto renderize a nuvem de pontos na posição correta e transmita ao HMD um fluxo de vídeo com os quadros renderizados. Na Seção 4, será considerada a segunda abordagem, para ter mais recursos computacionais no lado da decodificação.

Figura 6: Aumentar o tamanho dos pontos renderizados permite um método de *upscaling* simples e rápido. O tamanho do ponto é ilustrado como um ponto à esquerda da pessoa. (a) Nuvem de pontos amostrada a 40%; (b) nuvem de pontos com upscaling. *(imagem omitida nesta tradução; consulte o artigo original)*

3.6 VMAF em vídeo volumétrico

No contexto de vídeos volumétricos, o VMAF é usado capturando a vista renderizada em um ponto de vista fixo ou usando traços de movimento capturados, o que permite alterar o ponto de vista durante o vídeo [45]. Em contraste com as métricas baseadas em PSNR, o VMAF produz assim uma métrica que corresponde às imagens como percebidas pelo usuário [16].

4 AVALIAÇÃO E DISCUSSÃO

Esta seção apresenta primeiro a configuração experimental, fornecendo todos os detalhes de hardware e de implementação. Em seguida, discutimos as métricas de avaliação consideradas. Depois, apresentamos e discutimos os resultados de vários experimentos relacionados ao framework proposto baseado em WebRTC, a saber: o atraso de captura, a escalabilidade do codificador, o uso de largura de banda, a qualidade VMAF, o throughput e a latência do WebRTC e, por fim, uma visão geral da latência total de ponta a ponta.

4.1 Configuração experimental

Realizamos os experimentos de latência de pré-processamento, escalabilidade e qualidade VMAF em uma máquina com as seguintes especificações: *CPU: Intel 12th Gen i7-1265u (4,8 GHz Turbo), RAM: 16 GB DDR4 (3200 MHz).* Os experimentos de throughput e de latência do WebRTC são realizados no imec Virtual Wall [22], um testbed com um grande número de nós bare-metal interconectados. Usamos vários nós bare-metal conectados com as seguintes especificações de hardware: *CPU: Intel Xeon E5520 (2,27 GHz), RAM: 12 GB DDR3 (1066 MHz), NIC: 1 Gb/s.* Aplicamos controle de tráfego nos nós comutadores para limitar a largura de banda disponível; nenhuma latência artificial é adicionada. A configuração completa é ilustrada na Figura 7.

Para avaliar as abordagens sugeridas com os mesmos dados, usamos uma sequência capturada de nuvens de pontos com 1800 quadros [33]. Essa sequência contém nuvens de pontos geradas a partir de uma única câmera, com sequências alternadas de pouco ou nenhum movimento e gestos com as mãos para imitar o possível comportamento durante uma conversa. Além disso, para obter resultados realistas, empregamos um dataset com traços 4G [47] em um subconjunto dos nossos experimentos, para simular largura de banda flutuante. Esse dataset é usado nos seguintes experimentos: escalabilidade, uso de largura de banda e qualidade VMAF. Dois exemplos de traços são mostrados na Figura 8.

Figura 7: Configuração de hardware usada no setup experimental. *(imagem omitida nesta tradução; consulte o artigo original)*

Figura 8: Dois exemplos de traços de largura de banda 4G. *(imagem omitida nesta tradução; consulte o artigo original)*

Para capturar essa sequência, usamos uma câmera de profundidade Intel RealSense D455 [23] a 30 FPS e resolução de 848×480 pixels. A potência do laser do sensor de profundidade foi aumentada ao seu valor máximo de 360 para melhorar a qualidade visual e reduzir o número de buracos na nuvem de pontos resultante. Os quadros são codificados com o codec Draco, que garante latência de codificação em tempo real [14].

As etapas de pré-processamento (remoção de fundo, amostragem e codificação) foram colocalizadas com o componente de captura, utilizando o mesmo hardware. No entanto, devido à necessidade de ter um controlador de congestionamento totalmente implementado, implementamos o servidor e o cliente WebRTC em aplicações separadas usando Golang [17] junto com o pacote Pion [35]. Tanto a aplicação de pré-processamento quanto a de renderização interagem com sua respectiva contraparte WebRTC por meio de um socket UDP para transferir os dados necessários. Optamos pelo GCC como implementação do controlador de congestionamento para estimar a largura de banda usando as mensagens de transport wide congestion control (TWCC) transmitidas pelos clientes. Para a retransmissão de pacotes perdidos, é usado um sistema baseado em reconhecimento negativo (NACK) [21].

Para os métodos de adaptação, utilizamos o polinômio representado na Figura 4 para estimar a taxa de amostragem necessária dos codificadores individuais e dos codificadores MDC baseados em taxa de bits. Para os codificadores MDC, usamos a implementação mostrada na Tabela 1 para dividir nossas nuvens de pontos em três descrições, que permitem sete representações de qualidade possíveis. As taxas de amostragem das descrições-base (15%, 25% e 60%) usadas nos experimentos permitem uma ampla gama de taxas de bits disponíveis e foram escolhidas pelas seguintes razões:

- A taxa de 15%, pois taxas menores resultam em degradação significativa da qualidade;
- A taxa de 60%, que permite uma qualidade visual maior, em comparação com taxas menores, ao usar o upscaling descrito na Seção 3.5;
- A taxa de 25%, para permitir que 100% dos pontos sejam transmitidos ao combinar as três descrições.

O FoV é dividido igualmente nas três categorias de qualidade mostradas na Figura 5. Os níveis de distância são determinados pela altura da nuvem de pontos dentro do viewport (80%–100% do viewport para alta qualidade, 50%–80% para média qualidade, 20–50% para baixa qualidade). A aplicação de renderização usa o motor de jogos Unity [44]. Para renderizar as nuvens de pontos, usamos o pacote Pcx [34], com o shader incluído para renderizar os vértices como pontos individuais. Os quadros renderizados pelo Unity são exibidos no HMD Meta Quest 2 [29].

4.2 Métricas de avaliação

Com base na discussão das Seções 2 e 3, usamos as seguintes métricas para avaliar o desempenho de cada componente. Primeiro, avaliamos a latência introduzida no componente de captura pela câmera Intel RealSense D455. Usamos uma aplicação separada para calcular o tempo entre a renderização de um relógio binário em um display e a captura da imagem renderizada.

Segundo, avaliamos a escalabilidade dos codificadores individuais e MDC usando várias configurações, cada uma com um número diferente de clientes simulados. Usamos o tempo de pré-processamento e os FPS alcançados para estimar quão bem os sistemas conseguem escalar com um número crescente de clientes.

Terceiro, avaliamos o uso de largura de banda com duas métricas. Em primeiro lugar, examinamos a largura de banda média necessária para transmitir uma sequência de nuvens de pontos a 30 FPS para as diferentes abordagens de adaptação. A segunda métrica mede a eficácia das nossas estratégias de codificação em aproveitar a largura de banda disponível, especificamente o erro entre as taxas de bits resultantes e a largura de banda disponível.

Quarto, a qualidade é avaliada usando a métrica VMAF a partir de um único ponto de vista fixo. Para obter uma estimativa precisa, empregamos os traços de largura de banda 4G de van der Hooft et al. [47] para gerar quadros de qualidades variadas. Além disso, para os codificadores MDC, também mostramos a pontuação VMAF alcançada por cada uma das sete representações.

Quinto, avaliamos o componente WebRTC observando o throughput e a latência alcançados com duas possíveis configurações de largura de banda do enlace, 50 Mbit/s e 100 Mbit/s.

Sexto, apresentamos um detalhamento da latência total de ponta a ponta da nossa implementação proposta em comparação com a codificação individual, para múltiplas configurações de clientes.

4.3 Atraso de captura da câmera

Foi realizada uma avaliação do atraso de captura introduzido pelo sistema de câmera, a fim de avaliar o impacto da resolução e da taxa de quadros. Esse experimento foi realizado usando a ferramenta de latência incluída no SDK [24]. A ferramenta renderiza um relógio binário na tela do display, que é então capturado pela câmera. O quadro capturado é processado dentro da aplicação com a biblioteca OpenCV [31], que extrai o valor do relógio binário do quadro e o compara com o valor atual. A Tabela 2 apresenta as medições de atraso para várias resoluções a 30 e 60 FPS. O desempenho da aplicação foi avaliado em dois sistemas operacionais distintos, Ubuntu 22.04 e Windows 11, utilizando configurações de hardware idênticas. Notavelmente, o Ubuntu 22.04 apresenta desempenho superior ao do Windows 11. A razão exata desse fenômeno é difícil de determinar, pois depende de um número variável de fatores, como diferenças de drivers e de chamadas de sistema. Além disso, a versão para Windows 11 também apresenta comportamento inconsistente, levando a um atraso crescente ao longo da execução da aplicação. Fica evidente na Tabela 2 que a taxa de quadros tem maior impacto sobre o atraso do que a resolução. No entanto, a adoção de 60 FPS exige significativamente mais largura de banda e recursos computacionais, dificultando a manutenção do desempenho em tempo real.

Resolução	FPS	Latência (ms)	
		Ubuntu 22.04	Windows 11
424x240	30	70	91
	60	39	64
640x480	30	80	98
	60	45	67
848x480	30	85	104
	60	46	65
1280x720	30	100	115
	60	n/d	n/d

Tabela 2: Análise do atraso de captura da câmera Intel RealSense, modelo D455, para os sistemas operacionais Ubuntu 22.04 e Windows 11.

4.4 Escalabilidade da codificação

A codificação constitui a contribuição predominante para a latência na etapa de pré-processamento do nosso pipeline. A latência associada às demais operações de pré-processamento, isto é, a remoção de fundo e a amostragem, é desprezível (média de 3 ms) em comparação com a da codificação. A latência de codificação depende de vários fatores, como o número de pontos, a densidade e as semelhanças de cor entre pontos vizinhos. A Figura 9 ilustra esse efeito, indicando que o tempo de codificação cresce linearmente com o tamanho da nuvem de pontos.

No contexto da codificação individual, uma representação de qualidade única precisa ser codificada para cada cliente. Assim, clientes adicionais aumentam o tempo total gasto no pré-processamento de um único quadro. Naturalmente, ao codificar os fluxos individuais em paralelo, esse impacto pode ser reduzido. A Tabela 3 mostra o impacto de usar cinco threads de trabalho (*workers*) com um número variável de clientes. Nessa configuração de sistema, esperamos que todos os clientes terminem de codificar antes de capturar o próximo quadro, o que resulta em uma taxa de quadros menor quando o número de clientes aumenta. Para garantir que o sistema permaneça em 30 FPS constantes, é imperativo escalar o número de threads de trabalho proporcionalmente ao número de clientes. Além disso, a implementação de um buffer circular para as threads de trabalho permite codificar quadros consecutivos enquanto alguns clientes ainda estão codificando o quadro anterior. Em uma máquina com as seguintes especificações: *CPU: Intel 12th Gen i7-1265u (4,8 GHz Turbo), RAM: 16 GB DDR4 (3200 MHz)* e com cinco clientes simulados, alcançamos tempos de pré-processamento semelhantes empregando apenas três threads, com média de 18,27 ms (desvio-padrão = 4,56 ms). Disso inferimos que não é necessária uma quantidade igual de workers para o número de clientes e que o número necessário de workers pode ser estimado com a seguinte equação:

$$n_{\text{threads}} = n_{\text{clientes}} \times \frac{\text{Tempo médio de codificação}}{\text{Tempo entre quadros}} \quad (1)$$

No entanto, sob a hipótese de uma única máquina com capacidade limitada de doze threads, o sistema ainda consegue atender apenas um número modesto de clientes. Além disso, como mostra a Figura 9, o tempo de codificação cresce linearmente com a taxa de amostragem solicitada, o que torna o número de workers necessários altamente correlacionado ao número de representações de qualidade solicitadas.

# Clientes	Latência (ms)		FPS
	Média	DP	
5	14,87	5,78	29,12
10	32,43	16,32	18,34
20	55,99	29,16	9,65
40	97,81	51,28	5,46

Tabela 3: Latência de codificação individual para várias composições de clientes usando cinco threads de codificação. CPU: Intel 12th Gen i7-1265u (4,8 GHz Turbo), RAM: 16 GB DDR4 (3200 MHz).

A codificação MDC alivia esse problema por ter um número fixo de descrições usadas para construir níveis de qualidade adicionais. Além disso, o uso de descrições amostradas reduz o número máximo de pontos usados em uma única operação de codificação, diminuindo o tempo total de codificação. Essas descrições são codificadas em paralelo para reduzir o tempo total de

codificação necessário. Como resultado, o tempo de codificação de um quadro fica restrito ao tamanho da maior descrição, em vez de depender do número de clientes. A Figura 10 mostra o desempenho de codificação aumentado ao aplicar paralelização. O MDC limita o número de threads de trabalho necessárias ao número de descrições, tornando-o mais adequado para um sistema com recursos computacionais reduzidos ou com um grande número de clientes. Ganhos de desempenho semelhantes são obtidos no lado do cliente ao aplicar decodificação paralela às descrições.

Figura 9: Escalonamento linear do tempo de codificação com a taxa de amostragem.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 10: As abordagens de codificação MDC são aprimoradas ao codificar as três descrições em paralelo, usando um worker para cada descrição.
(imagem omitida nesta tradução; consulte o artigo original)

Comparando o tempo médio de pré-processamento (30 iterações, sequência de 1800 quadros) do codificador individual com o do codificador MDC, observamos que o codificador individual (cinco clientes, cinco workers) atinge um tempo médio de 14,87 ms (desvio-padrão = 5,78 ms), contra o codificador MDC (três descrições, três workers), que alcança uma média de 16,16 ms (desvio-padrão = 1,82 ms). Evidentemente, o tempo de pré-processamento da abordagem de codificação individual sofre muito com a ampla faixa de taxas de amostragem possíveis, pois as taxas mais altas aumentam significativamente o tempo máximo de codificação, criando um tempo de pré-processamento muito mais variável. Uma abordagem para mitigar esse problema seria a implementação de MDC no método de adaptação individual. Uma estratégia viável poderia consistir em impor um limite que permita codificar apenas 50% da nuvem de pontos em uma única thread, enquanto o restante dos pontos é codificado em uma thread separada. Além disso, isso permitiria reutilizar a descrição de 50% em vários clientes, reduzindo ainda mais o tempo de codificação em favor da eficiência de compressão.

4.5 Uso de largura de banda

O codificador MDC produz sete representações possíveis para cada quadro, criadas a partir de três descrições-base que correspondem a aproximadamente 15%, 25% e 60% dos pontos da nuvem original. A Tabela 4 mostra as taxas de bits resultantes para cada abordagem MDC. Fica claro, por essa tabela, que a abordagem MDC atende a uma faixa de larguras de banda variadas entre 12 Mbit/s e 75 Mbit/s. Para nos proteger contra uma subestimação da taxa de amostragem necessária e contra o fato de o estimador de largura de banda não se atualizar instantaneamente, optamos por usar uma salvaguarda que limita a taxa de bits disponível ao codificador a 90% da largura de banda estimada. Sem essa salvaguarda, o uso de largura de banda fica acima de 100%, causando perda de pacotes intolerável. Embora os métodos MDC nunca forneçam uma taxa de bits maior que a estimativa, optamos por usar a mesma salvaguarda para nos proteger contra a atualização não instantânea. Ao aplicá-la aos traços de largura de banda de van der Hooft et al., o codificador individual alcança um uso de largura de banda de 90,4% da largura de banda estimada, o que resulta em uma média de 3,17 Mbit/s de largura de banda desperdiçada. Por terem apenas um número limitado de representações, as abordagens MDC atingem um uso médio de largura de banda de apenas 75%, o que equivale a 7,88 Mbit/s desperdiçados. A Figura 11 mostra a taxa de bits média resultante de cada abordagem de codificação ao empregar os traços 4G.

	15%	**25%**	**40%**	**60%**	**75%**	**85%**	**100%**
Tamanho fixo	11,9	18,6	30,6	39,9	51,8	58,5	70,5
Taxa de bits fixa	12,6	19,6	32,1	41,2	53,6	60,8	73,3
Porcentagem	12,7	19,8	32,0	42,4	55,0	62,1	74,8

Tabela 4: Taxas de bits resultantes (Mbit/s) para as diferentes representações de qualidade MDC, sendo 15%, 25% e 60% as descrições-base usadas para gerar as demais representações.

Figura 11: Taxas de bits resultantes para todas as abordagens de codificação.
(imagem omitida nesta tradução; consulte o artigo original)

4.6 Qualidade VMAF

O VMAF nos permite medir o impacto de aumentar o tamanho dos pontos para aumentar a qualidade percebida. A Tabela 5 apresenta os resultados para tamanhos de ponto variados em todas as representações de qualidade possíveis obtidas do codificador MDC. A comparação é feita entre quadros amostrados e quadros renderizados da nuvem de pontos original. Ao renderizar a nuvem original, adotou-se um tamanho de ponto de 0,3 (unidades do Unity), pois valores menores introduzem lacunas na nuvem de pontos quando vista a partir do ponto de vista fixo. Pela Tabela 5, percebemos o impacto significativo de aumentar o tamanho dos pontos sobre a qualidade resultante. No entanto, é imperativo ter cautela para aumentar o tamanho dos pontos apenas quando necessário, a fim de evitar comprometer a qualidade. O impacto disso é notavelmente visível nas representações de maior qualidade, em que ocorre uma queda aproximada de 30% na qualidade ao dobrar o tamanho do ponto. Para o codificador individual, empregamos uma metodologia de upscaling semelhante. Entretanto, para esse codificador escolhemos o tamanho de ponto MDC cuja taxa de amostragem é a mais próxima da taxa de amostragem individual. Por exemplo, se um cliente individual requer uma taxa de amostragem de 20%, é escolhido um tamanho de ponto de 0,06 e, para uma taxa de amostragem de 43%, é selecionado um tamanho de ponto de 0,05. A Tabela 6 mostra a pontuação VMAF alcançada por cada uma das sete representações de qualidade. Observamos que as três variações oferecem desempenho semelhante no que diz respeito à qualidade visual do FoV resultante. Ao comparar com as pontuações VMAF da abordagem de codificação individual, para um número variável de clientes, podemos ver que a abordagem baseada em MDC pontua melhor quando há pelo menos vinte clientes envolvidos (ver Tabela 7, sequência de 1800 quadros). Esse fenômeno é explicado pelo fato de a sequência de nuvens de pontos usada ter segmentos com pouco ou nenhum movimento, que resultam em uma pontuação VMAF alta, mesmo com taxas de quadros menores. O desvio-padrão dos experimentos abaixo também indica isso. A partir dos dados da Tabela 7, concluímos que a abordagem MDC funciona bem em ambientes com muitos clientes ou para vídeos volumétricos que sofrem com baixa taxa de quadros.

Tabela 5: Pontuações VMAF para todas as combinações possíveis de qualidade e tamanho, para tamanhos de ponto de 0,03 a 0,09.

	15%		25%		40%		60%		75%		85%		100%	
	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP
0,03	4,99	2,72	24,06	2,93	47,38	2,68	68,58	2,17	78,69	1,6	83,42	1,26	88,06	0,84
0,04	22,03	2,93	44,46	2,66	65,28	1,87	75,82	1,08	77,65	0,82	77,74	0,73	77,18	0,75
0,05	33,98	2,85	55,26	2,13	67,56	1,14	68,98	1,03	67,9	1,01	67,0	1,0	65,64	1,06
0,06	41,53	2,58	57,22	1,44	61,16	1,11	59,05	1,22	57,38	1,21	56,46	1,32	55,18	1,3
0,07	44,72	2,08	52,31	1,49	50,88	1,34	47,71	1,5	45,96	1,58	44,99	1,56	43,84	1,58
0,08	42,89	1,68	44,32	1,54	40,81	1,58	37,66	1,72	36,02	1,75	35,14	1,78	34,12	1,85
0,09	37,87	1,58	35,82	1,65	31,5	1,72	28,7	1,83	27,34	1,99	26,52	1,85	25,66	1,9

Tabela 6: Pontuações VMAF das abordagens MDC com qualidade fixa.

	15%		25%		40%		60%		75%		85%		100%	
	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP
Tam. fixo	44,52	2,13	57,05	1,7	67,69	1,16	75,84	1,11	78,73	1,58	83,24	1,31	88,05	0,8
Taxa fixa	49,99	1,84	61,01	1,29	69,35	1,04	72,98	1,42	76,25	1,57	83,87	1,16	89,32	0,61
Porcent.	46,62	1,93	58,64	1,48	68,61	1,08	76,71	0,82	81,71	0,69	85,9	0,59	89,76	0,47

			Pontuação VMAF	
	# Clientes	FPS	Média	DP
Individual adaptativa	5	30	67,98	11,93
	10	20	67,67	13,50
	20	10	65,90	16,48
	40	5	58,47	19,20
MDC tamanho fixo	n	30	65,98	9,01
MDC taxa de bits fixa	n	30	65,12	11,63
MDC porcentagem	n	30	66,64	7,39

Tabela 7: Pontuações VMAF de uma sequência de 1800 quadros para ambas as abordagens baseadas em MDC contra a codificação individual, usando três descrições nas abordagens baseadas em MDC.

4.7 Throughput e latência do WebRTC

O objetivo do nosso pipeline é conseguir adaptar seu conteúdo com base na largura de banda disponível, mantendo ao mesmo tempo uma latência adequada à comunicação em tempo real. Para testar o throughput alcançável e a latência do WebRTC, usamos duas capacidades de enlace diferentes, 50 Mbit/s e 100 Mbit/s. Justificamos o uso da capacidade de 50 Mbit/s por ela estar abaixo da largura de banda máxima necessária, mas ainda permitir um número suficiente de representações de qualidade. Neste experimento, empregamos o codificador MDC de tamanho fixo por ele apresentar as taxas de bits resultantes mais consistentes, já que uma largura de banda estável facilita a convergência do controlador de congestionamento. A Figura 12 mostra os resultados deste experimento; observamos que o enlace de 100 Mbit/s atinge uma taxa de bits estável de 70,6 Mbit/s. Pela Tabela 4, observamos que isso corresponde à taxa de bits da representação de maior qualidade. Em contraste, o enlace de 50 Mbit/s se estabiliza em 30,6 Mbit/s, correspondente à representação de 40%. No entanto, mesmo com a salvaguarda de 90%, esse enlace deveria conseguir atingir a taxa de bits da representação de 60%. A Figura 12 indica que esse comportamento se deve ao fato de o algoritmo GCC não conseguir se recuperar após uma superestimação que causou perda de pacotes. Analisando ambas as figuras, podemos concluir que o algoritmo GCC leva um tempo significativo até convergir para uma estimativa estável. Esse fenômeno está relacionado à taxa de bits inicial do GCC, que foi definida como a largura de banda mínima necessária. Aumentar a taxa de bits inicial acelera a convergência ao custo de causar perda de pacotes em enlaces de baixa largura de banda.

No caso do enlace de 50 Mbit/s, a convergência é observada em torno de 30 Mbit/s. Conforme a Tabela 4, isso corresponde a um nível de qualidade de 40%, com pontuação VMAF média de 67,56.

Em termos de latência, o enlace de 100 Mbit/s alcança uma latência de transporte média de 30,89 ms (desvio-padrão = 10,22 ms), em comparação com o resultado do enlace de 50 Mbit/s, que alcança 24,42 ms (desvio-padrão = 5,68 ms). Ambos os resultados são aceitáveis para uso em um ambiente de tempo real. A menor latência do enlace de 50 Mbit/s é atribuída ao menor throughput e indica que o algoritmo GCC garante um bom equilíbrio entre largura de banda estimada, latência e possível perda de pacotes.

Figura 12: Throughput obtido ao usar a abordagem MDC de tamanho fixo para 50 Mbit/s e 100 Mbit/s.
(imagem omitida nesta tradução; consulte o artigo original)

4.8 Detalhamento da latência de ponta a ponta

A Figura 13 ilustra a latência completa de ponta a ponta para ambas as abordagens de codificação, sem atrasos de enlace adicionados artificialmente. Para este experimento, usamos a sequência de quadros fixa com três workers. No entanto, adicionamos a latência da câmera por completude. Implantamos duas instâncias da aplicação cliente em cada nó ao aumentar o número de clientes para oito. A cada cliente são atribuídos 100 Mbit/s de largura de banda usando tc.

Evidentemente, o componente de captura tem a influência mais substancial, constituindo entre 50% e 64% da latência total. No entanto, o uso de uma câmera mais otimizada traria melhorias imediatas no componente de captura. No geral, observamos que nosso pipeline é capaz de transmitir nuvens de pontos capturadas a uma taxa de 30 FPS e resolução de 848x480 pixels com latência baixa o suficiente para viabilizar comunicação em tempo real. Para um número maior de clientes, a abordagem baseada em MDC consegue manter uma latência estável em comparação com a codificação individual, cujo tempo de codificação aumenta devido ao número limitado de workers.

Figura 13: Latência composta de cada componente descrito na arquitetura proposta, sem atrasos de enlace artificiais, usando três threads de trabalho.
(imagem omitida nesta tradução; consulte o artigo original)

5 CONCLUSÃO E TRABALHOS FUTUROS

Neste artigo, propusemos uma nova arquitetura um-para-muitos para viabilizar o streaming de vídeos volumétricos. Realizamos uma análise comparativa de diversas estratégias de codificação e adaptação. Nossas constatações indicam que uma representação de qualidade codificada individualmente para cada cliente produz os melhores resultados para um número limitado de clientes. Para tratar as preocupações de escalabilidade, introduzimos uma abordagem baseada em MDC que utiliza várias descrições distintas para construir múltiplas representações de qualidade. Apesar de apresentar utilização de largura de banda ideal e qualidade objetiva comparativamente inferiores, a abordagem MDC demonstra escalabilidade superior, alcançando tempos de pré-processamento equivalentes com um número substancialmente menor de workers. Em comparação com a codificação por cliente, observamos latência de ponta a ponta 9% menor (163 ms contra 182 ms) com três clientes e 19% menor (166 ms contra 204 ms) ao aumentar o número de clientes para oito. Concluímos que nosso pipeline é capaz de produzir uma latência aceitável com qualidade visual comparável à do codificador individual em ambientes com poucos clientes, e é capaz de produzir qualidade média superior em ambientes com mais de 20 clientes.

Em trabalhos futuros, focaremos em estender a arquitetura para permitir streaming muitos-para-muitos. Além disso, a extensão para múltiplos clientes de entrada exigirá que aprimoremos os algoritmos de codificação e de adaptação, a fim de alcançar taxas de bits e qualidade visual aceitáveis em cenas com um grande número de clientes. Também será feita uma extensão para permitir o uso de múltiplas câmeras, que será avaliada por meio de pontuações VMAF para múltiplos pontos de vista.

AGRADECIMENTOS

Este trabalho foi financiado pela União Europeia (projeto SPIRIT, Acordo de Subvenção 101070672, https://www.spirit-project.eu/).

REFERÊNCIAS (NO IDIOMA ORIGINAL)

[1] S. J. Ahn, L. Levy, A. Eden, A. S. Won, B. MacIntyre, and K. Johnsen. 2021. IEEEVR2020: Exploring the first steps toward standalone virtual conferences. Frontiers in Virtual Reality, 2, 648575.
[2] D. S. Alexiadis, D. Zarpalas, and P. Daras. 2012. Real-time, full 3D reconstruction of moving foreground objects from multiple consumer depth cameras. IEEE Transactions on Multimedia, 15, 2, 339–358.
[3] M. A. Azad, R. Mahmood, and T. Mehmood. 2009. A comparative analysis of DCCP variants (CCID2, CCID3), TCP and UDP for MPEG4 video applications. In 2009 International Conference on Information and Communication Technologies. IEEE, 40–45.
[4] S.-H. Bae, J. Kim, M. Kim, S. Cho, and J. S. Choi. 2013. Assessments of subjective video quality on HEVC-encoded 4K-UHD video for beyond-HDTV broadcasting services. IEEE Transactions on Broadcasting, 59, 2, 209–222.
[5] M. Bassier, M. Vergauwen, and F. Poux. 2020. Point cloud vs. mesh features for building interior classification. Remote Sensing, 12, 14, 2224.
[6] N. Bouzakaria, C. Concolato, and J. Le Feuvre. 2014. Overhead and performance of low latency live streaming using MPEG-DASH. In IISA 2014, The 5th International Conference on Information, Intelligence, Systems and Applications. IEEE, 92–97.
[7] M. Broxton et al. 2020. Immersive light field video with a layered mesh representation. ACM Transactions on Graphics (TOG), 39, 4, 86–1.
[8] K. Cao, Y. Xu, and P. Cosman. 2020. Visual quality of compressed mesh and point cloud sequences. IEEE Access, 8, 171203–171217.
[9] N. Cardwell, S. Savage, and T. Anderson. 2000. Modeling TCP latency. In Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No. 00CH37064). Vol. 3. IEEE, 1742–1751.
[10] G. Carlucci, L. De Cicco, S. Holmer, and S. Mascolo. 2016. Analysis and design of the google congestion control for web real-time communication (WebRTC). In Proceedings of the 7th International Conference on Multimedia Systems, 1–12.
[11] Y. Cui, S. Schuon, D. Chan, S. Thrun, and C. Theobalt. 2010. 3D shape scanning with a time-of-flight camera. In 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. IEEE, 1173–1180.
[12] E. d'Eon, B. Harrison, T. Myers, and P. A. Chou. 2017. 8i voxelized full bodies-a voxelized point cloud dataset. ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG) input document WG11M40059/WG1M74006, 7, 8, 11.
[13] Q. Dai and R. Lehnert. 2010. Impact of packet loss on the perceived video quality. In 2010 2nd International Conference on Evolving Internet. IEEE, 206–209.
[14] M. De Fré, F. De Turck, and J. van der Hooft. 2023. Master Thesis Dissertation: Low-Latency Volumetric-Video Delivery for Real-Time Conferencing. (2023). http://lib.ugent.be/catalog/rug01:003150149.
[15] C. Debeunne and D. Vivet. 2020. A review of visual-LiDAR fusion based simultaneous localization and mapping. Sensors, 20, 7, 2068.
[16] B. García, L. López-Fernández, F. Gortázar, and M. Gallego. 2019. Practical evaluation of VMAF perceptual video quality for WebRTC applications. Electronics, 8, 8, 854.
[17] [SW], Golang 2023. url: https://go.dev/.
[18] M. J. Gourlay and R. T. Held. 2017. Head-mounted-display tracking for augmented and virtual reality. Information Display, 33, 1, 6–10.
[19] D. Graziosi, O. Nakagami, S. Kuma, A. Zaghetto, T. Suzuki, and A. Tabatabai. 2020. An overview of ongoing point cloud compression standardization activities: Video-based (V-PCC) and geometry-based (G-PCC). APSIPA Transactions on Signal and Information Processing, 9, e13.
[20] J. Hacker, J. vom Brocke, J. Handali, M. Otto, and J. Schneider. 2020. Virtually in this together–how web-conferencing systems enabled a new virtual togetherness during the covid-19 crisis. European Journal of Information Systems, 29, 5, 563–584.
[21] S. Holmer, M. Shemer, and M. Paniconi. 2013. Handling packet loss in WebRTC. In 2013 IEEE International Conference on Image Processing. IEEE, 1860–1864.
[22] [SW], imec Virtual Wall 2023. url: https://doc.ilabt.imec.be/ilabt/virtualwall/.
[23] [SW], Intel Realsense 2023. url: https://www.intelrealsense.com/.
[24] [SW], Intel RealSense SDK 2 2023. url: https://www.intelrealsense.com/sdk-2/.
[25] J. Jansen, S. Subramanyam, R. Bouqueau, G. Cernigliaro, M. M. Cabré, F. Pérez, and P. Cesar. 2020. A pipeline for multiparty volumetric video conferencing: transmission of point clouds over low latency dash. In Proceedings of the 11th ACM Multimedia Systems Conference, 341–344.
[26] I. Johansson. 2014. Self-clocked rate adaptation for conversational video in LTE. In Proceedings of the 2014 ACM SIGCOMM workshop on Capacity sharing workshop, 51–56.
[27] A. B. Johnston and D. C. Burnett. 2012. WebRTC: APIs and RTCWEB protocols of the HTML5 real-time web. Digital Codex LLC.
[28] L. Keselman, J. Iselin Woodfill, A. Grunnet-Jepsen, and A. Bhowmik. 2017. Intel realsense stereoscopic depth cameras. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops, 1–10.
[29] [SW] Meta, Meta Quest 2 Virtual Reality Headset 2023. url: https://www.meta.com/quest/products/quest-2.
[30] M. Nguyen, S. Vats, S. Van Damme, J. van der Hooft, M. Torres Vega, T. Wauters, C. Timmerer, and H. Hellwagner. 2023. Impact of quality and distance on the perception of point clouds in mixed reality. In 2023 15th International Conference on Quality of Multimedia Experience (QoMEX). IEEE, 87–90.
[31] [SW], OpenCV 2023. url: https://opencv.org/.
[32] [SW], MDC-Based WebRTC Point Cloud Streaming 2023. url: https://github.com/MatthiasDeFre/webrtc-pc-streaming.
[33] [SW], Point Cloud Sequence Used in Experiments 2023. url: https://users.ugent.be/~madfr/pc-streaming-mmsys2024/frames.zip.
[34] [SW] keijiro, https://www.meta.com/be/en/quest/products/quest-2/ 2023. url: https://github.com/keijiro/Pcx.
[35] [SW], Pion WebRTC 2023. url: https://github.com/pion/webrtc.
[36] R. Rassool. 2017. VMAF reproducibility: Validating a perceptual practical video quality metric. In 2017 IEEE international symposium on broadband multimedia systems and broadcasting (BMSB). IEEE, 1–2.
[37] R. Rejaie, M. Handley, and D. Estrin. 1999. Quality adaptation for congestion controlled video playback over the internet. ACM SIGCOMM Computer Communication Review, 29, 4, 189–200.
[38] D. Roberts, T. Duckworth, C. Moore, R. Wolff, and J. O'Hare. 2009. Comparing the end to end latency of an immersive collaborative environment and a video conference. In 2009 13th IEEE/ACM International Symposium on Distributed Simulation and Real Time Applications. IEEE, 89–94.

[39] O. Schreer, I. Feldmann, S. Renault, M. Zepp, M. Worchel, P. Eisert, and P. Kauff. 2019. Capture and 3D video processing of volumetric video. In 2019 IEEE International conference on image processing (ICIP). IEEE, 4310–4314.
[40] M. Schütz et al. 2016. Potree: rendering large point clouds in web browsers. Technische Universität Wien, Wiedeń.
[41] M. Seufert, S. Egger, M. Slanina, T. Zinner, T. Hoßfeld, and P. Tran-Gia. 2014. A survey on quality of experience of HTTP adaptive streaming. IEEE Communications Surveys & Tutorials, 17, 1, 469–492.
[42] D. Tian, H. Ochimizu, C. Feng, R. Cohen, and A. Vetro. 2017. Geometric distortion metrics for point cloud compression. In 2017 IEEE International Conference on Image Processing (ICIP). IEEE, 3460–3464.
[43] E. M. Torlig, E. Alexiou, T. A. Fonseca, R. L. de Queiroz, and T. Ebrahimi. 2018. A novel methodology for quality assessment of voxelized point clouds. In Applications of Digital Image Processing XLI. Vol. 10752. SPIE, 174–190.
[44] [SW], Unity 2023. url: https://unity.com.
[45] S. Van Damme, M. Torres Vega, and F. De Turck. 2021. A full-and no-reference metrics accuracy analysis for volumetric media streaming. In 2021 13th International Conference on Quality of Multimedia Experience (QoMEX). IEEE, 225–230.
[46] J. van der Hooft, H. Amirpour, M. Torres Vega, Y. Sanchez, R. Schatz, T. Schierl, and C. Timmerer. 2023. A tutorial on immersive video delivery: from omnidirectional video to holography. IEEE Communications Surveys & Tutorials.
[47] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. Rondao Alface, T. Bostoen, and F. De Turck. 2016. HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks. IEEE Communications Letters, 20, 11, 2177–2180.
[48] J. van der Hooft, T. Wauters, F. De Turck, C. Timmerer, and H. Hellwagner. 2019. Towards 6dof http adaptive streaming through point cloud compression. In Proceedings of the 27th ACM International Conference on Multimedia, 2405– 2413.
[49] Wowza. 2021. 2021 Video Streaming Latency Report. Tech. rep. Wowza.
[50] M. Yang, Z. Luo, M. Hu, M. Chen, and D. Wu. 2023. A comparative measurement study of point cloud-based volumetric video codecs. IEEE Transactions on Broadcasting.
[51] E. Zerman, C. Ozcinar, P. Gao, and A. Smolic. 2020. Textured mesh vs coloured point cloud: a subjective study for volumetric video compression. In 2020 Twelfth International Conference on Quality of Multimedia Experience (QoMEX). IEEE, 1–6.
[52] Y. Zhang, Z. Li, S. Xu, C. Li, J. Yang, X. Tong, and B. Guo. 2023. RemoteTouch: Enhancing Immersive 3D Video Communication with Hand Touch. In 2023 IEEE Conference Virtual Reality and 3D User Interfaces (VR). IEEE, 1–10.
[53] D. J. Zielinski, H. M. Rao, M. A. Sommer, and R. Kopper. 2015. Exploring the effects of image persistence in low frame rate virtual environments. In 2015 IEEE Virtual Reality (VR). IEEE, 19–26.